# An improved schedule for LeanCoP options

(aka "the PAAR and/or AITP paper")

## Introduction and Research

LeanCoP inspired a large family of connection provers that improve performance and completeness by making a small change to the control mechanism [6].

LeanCoP 2.0 [7] added 3 control options to the original LeanCoP: `cut` and `scut`, to restrict backtracking in connection calculus, and `comp(J)`, which makes iterative deepening complete by restarting the search and increasing the maximum height of this tree 1 at a time until it reaches maximum height `J`, and restarting from height 1 and with cuts disabled after exhausting the final tree.

This work gathers ideas from other implementations of connection provers from different domains.

Letz and Stenz [4] show that, in tableau search, it's preferable to have subgoals with fewer solutions to detect dead ends earlier. Reordering clauses to prioritise shorter clauses is a valid principle, because the number of solutions is roughly the product over the clause's subgoals.

Zombori, Urban, and Olšák [9] show that the entropy of a learned policy's action distribution is a key factor in guiding connection provers: policies whose entropy is too low tend to overcommit to narrow search paths, while policies whose entropy is too high spread effort too thinly. I adapt this insight at runtime by measuring the Shannon entropy of heuristic action scores at each proof step and use low entropy as a signal to add a `cut` to the current connection action.

Li et al. [5] frame the reset choice in SAT solvers as a multi-armed bandit problem. They show that recent search productivity should influence reset behaviour. Similar restart strategies for SAT solvers are also surveyet in Holden [2].

## Novel Idea

Triggering the three control atoms in LeanCoP, `cut`, `scut`, and `comp(I)`, dynamically during the tree search rather than at the beginning can allow a prover to solve a tableau tree in less time and fewer steps.

A possible solution would be to use reinforcement learning, like RLCop [3] or learnCoP [citation coming soon!]. However, inference in these systems is slow and a heuristic-based solution might solve problems more quickly on a subset of TPTP problems.

## Implementation

I use the existing connection prover implementation as a base for this code [8]. This allows me to use the existing code for hard things like Prolog parsing, and also to use PyCoP as a control group.

My code implements 6 different strategies.

**FirstAction** *(control)*
: Run the first legal action (PyCoP default).

**LeanCoPCut** *(control)*
: Run `cut`, `scut`, and `comp(7)`, like LeanCoP 2.0.

**HeuristicSelection**
: Choose reductions if possible, otherwise choose the extension with the shortest clause.

**EntropyCut**
: Run the first legal action, but add a cut when heuristic action scores have low entropy.

**AdaptiveReset**
: Resets the tree if it hits 5 dead-ends in a row.

**SmartCut**
: Combines the last three strategies.

## Results

Experiments were run on the University of Cambridge CSD3 Ice Lake cluster, using one node per experiment with 76 physical CPU cores and 256 GiB RAM. For comparative transparency, both the control and test groups were run with the same backend [8].

A subset of 247 "representative" TPTP problems was used for evaluation [1]. The results with any significance are found in Table 1.

The comparer runs all 6 policies in parallel, with a 30-second time limit and a 50 000 step limit. A larger

| Problem | FirstAction | LeanCopCut | HeuristicSelection | EntropyCut | AdaptiveReset | SmartCut |
|---|---|---|---|---|---|---|
| GEO179+3 | 11 987 | 4744 | 9629 | 11 979 | 2546 | max-steps |
| GEO181+3 | 11 929 | 4778 | 9812 | 11 925 | max-steps | timeout |
| HWV026-3 | 10 174 | 4212 | max-steps | 10 174 | 10 174 | max-steps |
| KRS095+1 | 552 | 504 | 952 | 548 | 552 | 1014 |
| LCL186-1 | 245 | 477 | 297 | 245 | 368 | timeout |
| LCL644+1.001 | 8 | 8 | 2 | 8 | 8 | 2 |
| LCL679+1.001 | 1 | 1 | 1 | 1 | 1 | 1 |
| MSC015-1.005 | 25 401 | 29 037 | 26 257 | 28 251 | 7556 | 8816 |
| NUM291+1 | 32 860 | 19 936 | max-steps | 32 860 | 32 860 | max-steps |
| NUM300+1 | 11 135 | 7471 | 18 308 | 11 135 | 11 135 | 18 306 |
| NUM615+1 | max-steps | 42 778 | max-steps | max-steps | max-steps | max-steps |
| NUM634+3 | timeout | timeout | 2 | timeout | timeout | 2 |
| SET062+3 | 24 | 24 | 26 | 24 | 24 | 26 |
| SET175+3 | max-steps | 12 401 | max-steps | max-steps | 1062 | 1031 |
| SET603+3 | max-steps | 9953 | max-steps | max-steps | timeout | 853 |
| SET626+3 | max-steps | 35 231 | max-steps | max-steps | timeout | timeout |
| SET886+1 | max-steps | 24 582 | max-steps | max-steps | timeout | timeout |
| SET904+1 | 105 | 101 | 140 | 101 | 105 | 140 |
| SEU121+2 | max-steps | 34 536 | max-steps | max-steps | timeout | timeout |
| SWV232+1 | 54 | 54 | timeout | 54 | 54 | timeout |
| SWV248-2 | timeout | timeout | 7016 | 6018 | timeout | timeout |
| SWV264-2 | 9 | 9 | 11 | 9 | 9 | 11 |
| SYN061-1 | 46 | 46 | 46 | 46 | 46 | 46 |
| SYN582-1 | 15 843 | 11 650 | 40 624 | 15 430 | timeout | timeout |
| SYN663-1 | 5651 | max-steps | 5588 | 5626 | timeout | max-steps |
| SYN709-1 | 16 642 | 5238 | 11 201 | 16 065 | max-steps | max-steps |
| SYN726-1 | max-steps | max-steps | 4468 | max-steps | timeout | max-steps |

**Table 1:** Amount of steps required to solve representative TPTP problems. Green numbers show best results that are significantly better than the control groups. Only 27 of these problems finished within the time and step limit; larger experiments with a higher time limit will give us more information.

experiment, with the entirety of TPTP and a longer time limit, will come in the future.

I also post-process the canonical TPTP runs into short serial schedules. For each strategy and time cap, the pair "strategy + cap" is treated as covering exactly the problems that the strategy solves within that cap. The schedule synthesis problem is therefore a budgeted maximum-coverage problem with a multiple-choice knapsack constraint: choose at most one cap per strategy, subject to a global budget of 60 or 120 seconds, and maximise either the total number of solved problems or the number of solved problems outside LeanCopCut's solved set. After fixing the capped strategies, I then order them by a second optimisation pass that minimises total runtime over the benchmark set, using the recorded per-problem runtimes and charging the full cap on timeout rows.

The results show that all three policies have marked improvements in certain problems. It's likely that with further exploration we might get even better results than the control groups.

# References


[1] Karel Chvalovský and Jan Jakubův. "Characteristic Subsets of TPTP Benchmarks". In: *Proceedings of the 6th Conference on Artificial Intelligence and Theorem Proving (AITP 2021)* (Aussois, France, Sept. 5–10, 2021). Extended abstract in informal proceedings. 2021.

[2] Sean B. Holden. "Machine Learning for Automated Theorem Proving: Learning to Solve SAT and QSAT". In: *Foundations and Trends® in Machine Learning* 14.6 (2021), pp. 807–989. ISSN: 1935-8237, 1935-8245.

[3] Cezary Kaliszyk et al. "Reinforcement Learning of Theorem Proving". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018, pp. 8836–8847.

[4] Reinhold Letz and Gernot Stenz. "Model Elimination and Connection Tableau Procedures". In: *Handbook of Automated Reasoning*. Elsevier, 2001, pp. 2015–2114. ISBN: 978-0-444-50813-3.

[5] Chunxiao Li et al. "A Reinforcement Learning Based Reset Policy for CDCL SAT Solvers". In: *CoRR*

abs/2404.03753 (Apr. 19, 2024). arXiv: 2404.03753 [cs.LO]. Pre-published.

[6] Jens Otten. "20 Years of leanCoP — an Overview of the Provers". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*. Vol. 3613. CEUR Workshop Proceedings. 2023.

[7] Jens Otten. "Restricting Backtracking in Connection Calculi". In: *AI Communications* 23.2–3 (2010), pp. 159–182. ISSN: 09217126.

[8] Fredrik Rømming, Jens Otten, and Sean B. Holden. "Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*. Vol. 3613. CEUR Workshop Proceedings. 2023, pp. 107–118.

[9] Zsolt Zombori, Josef Urban, and Miroslav Olšák. "The Role of Entropy in Guiding a Connection Prover". In: *Automated Reasoning with Analytic Tableaux and Related Methods*. Ed. by Anupam Das and Sara Negri. Vol. 12842. Cham: Springer International Publishing, 2021, pp. 218–235. ISBN: 978-3-030-86058-5 978-3-030-86059-2.